Week 4: Aggregates and joins

Aggregate functions

Function	Returns
`COUNT(*)`	Number of rows
`COUNT(col)`	Number of rows where `col` is **not NULL**
`SUM(col)` `AVG(col)`	Total / mean (NULLs ignored)
`MIN(col)` `MAX(col)`	Smallest / largest
`ROUND(x, 1)`	Round to 1 decimal place

```
SELECT COUNT(*) AS students,
       ROUND(AVG(score), 1) AS average
FROM scores;
```

GROUP BY: one result row per group

```
SELECT class, COUNT(*) AS n
FROM students
GROUP BY class
ORDER BY n DESC;
```

- Every column in `SELECT` must be **grouped** or **inside an aggregate**.
- `WHERE` filters **rows** before grouping. `HAVING` filters **groups** after.

```
SELECT subject_id, AVG(score) AS avg
FROM scores
WHERE score IS NOT NULL      -- rows
GROUP BY subject_id
HAVING AVG(score) > 66;      -- groups
```

How SQL really reads a query

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT. That is why a column alias cannot be used in `WHERE`.

JOIN: combine tables

```
SELECT s.name, su.name AS subject, sc.score
FROM scores AS sc
JOIN students AS s  ON s.student_id  = sc.student_id
JOIN subjects AS su ON su.subject_id = sc.subject_id
WHERE sc.score >= 90;
```

- `ON` says how rows match: usually **foreign key = primary key**.
- Give tables short **aliases** (`AS s`) and write `s.name` so it is clear which `name`.
- Forget the `ON`, and you get every combination of rows (a "cross join").

Join	Keeps
`JOIN` (inner)	Only rows that match on both sides
`LEFT JOIN`	**All** rows from the left table; NULLs where there is no match

Find rows with no match (anti-join)

```
SELECT st.name
FROM students AS st
LEFT JOIN student_cca AS sc
       ON sc.student_id = st.student_id
WHERE sc.student_id IS NULL;   -- in no CCA
```

Count with LEFT JOIN

```
SELECT c.name, COUNT(sc.student_id) AS members
FROM ccas AS c
LEFT JOIN student_cca AS sc ON sc.cca_id = c.cca_id
GROUP BY c.cca_id;
```

> Count a column from the **right** table (`COUNT(sc.student_id)`), not `COUNT(*)`. `COUNT(*)` would count the empty club as 1.

Subqueries

```
SELECT name FROM students
WHERE student_id IN
  (SELECT student_id FROM scores WHERE score = 100);

SELECT name, score FROM scores
WHERE score > (SELECT AVG(score) FROM scores);
```